

Data imbalance problem, Data visualization

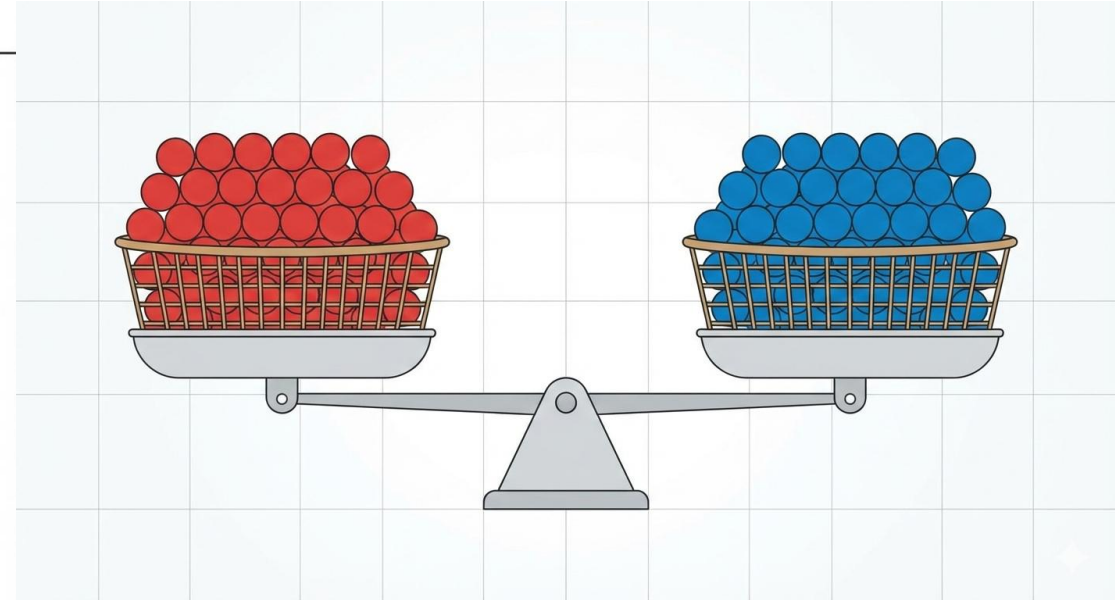
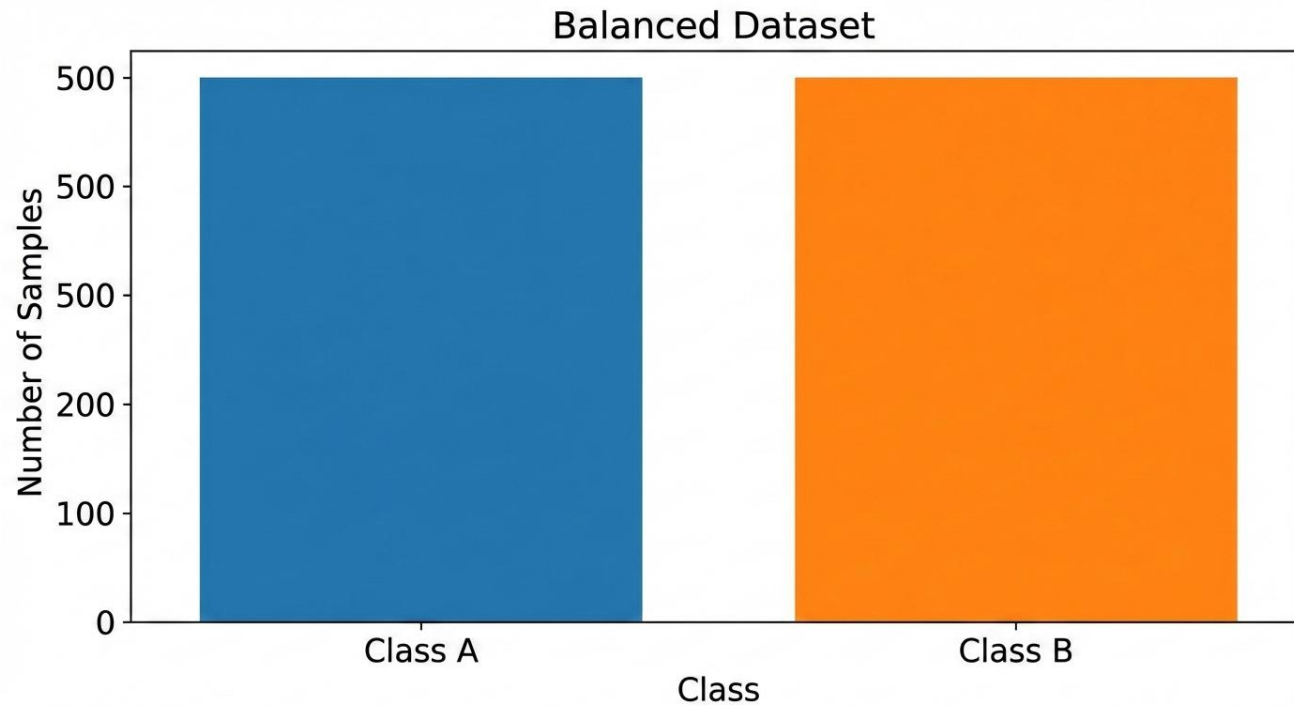
Dr Gayathri M
ECE



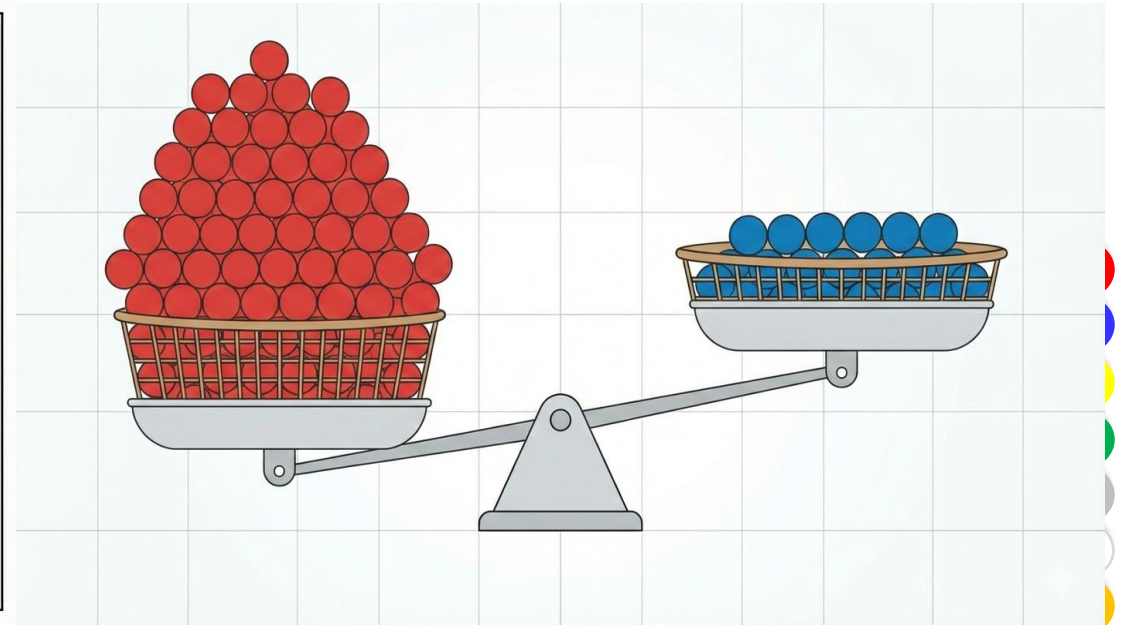
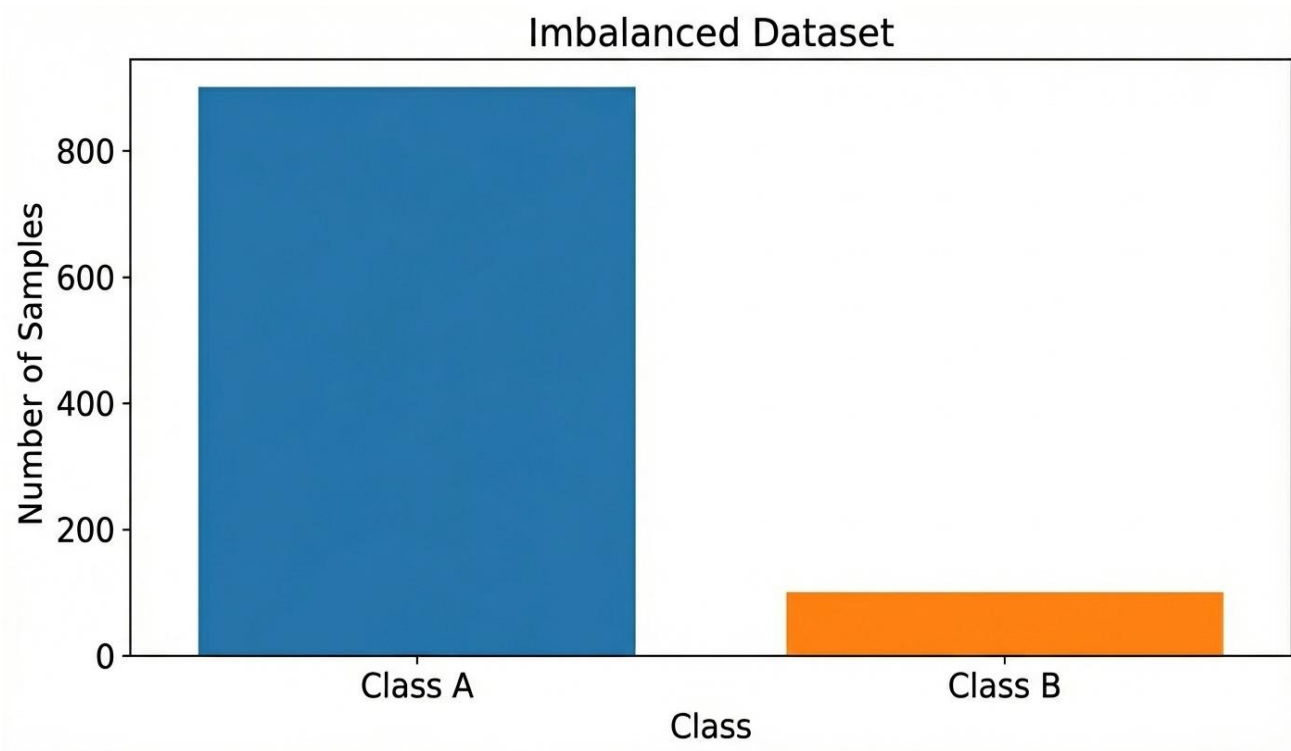
Imbalanced Datasets



Balance dataset



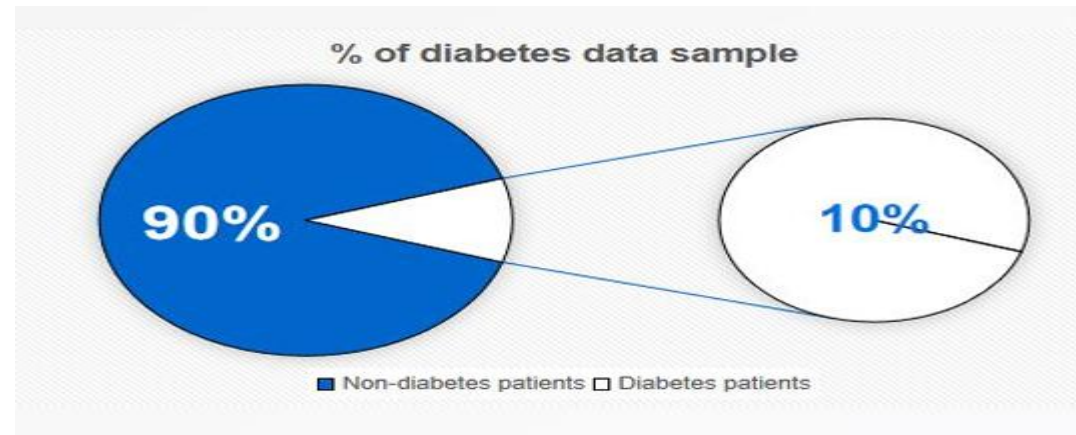
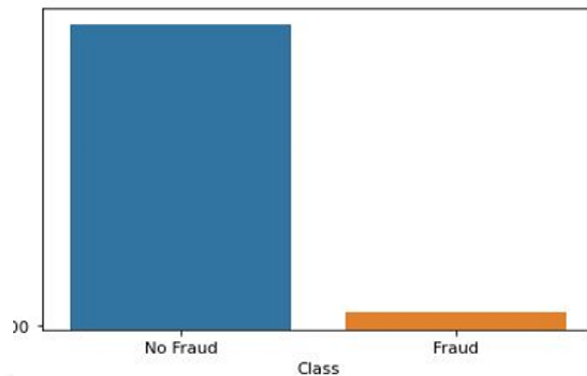
Imbalance dataset



Data Imbalance Problem

- Imbalanced data is a common problem in machine learning
- In machine learning, “imbalanced classes” is a familiar problem particularly occurring in classification when we have datasets with an unequal ratio of data points in each class.

Imbalanced Data Examples



Imbalanced data in practice

Example

You are a bank employee responsible for detecting the validity of credit card transactions. To do so, you have a training set of previously observed transactions, each of which was either:

a) Normal

b) Fraudulent



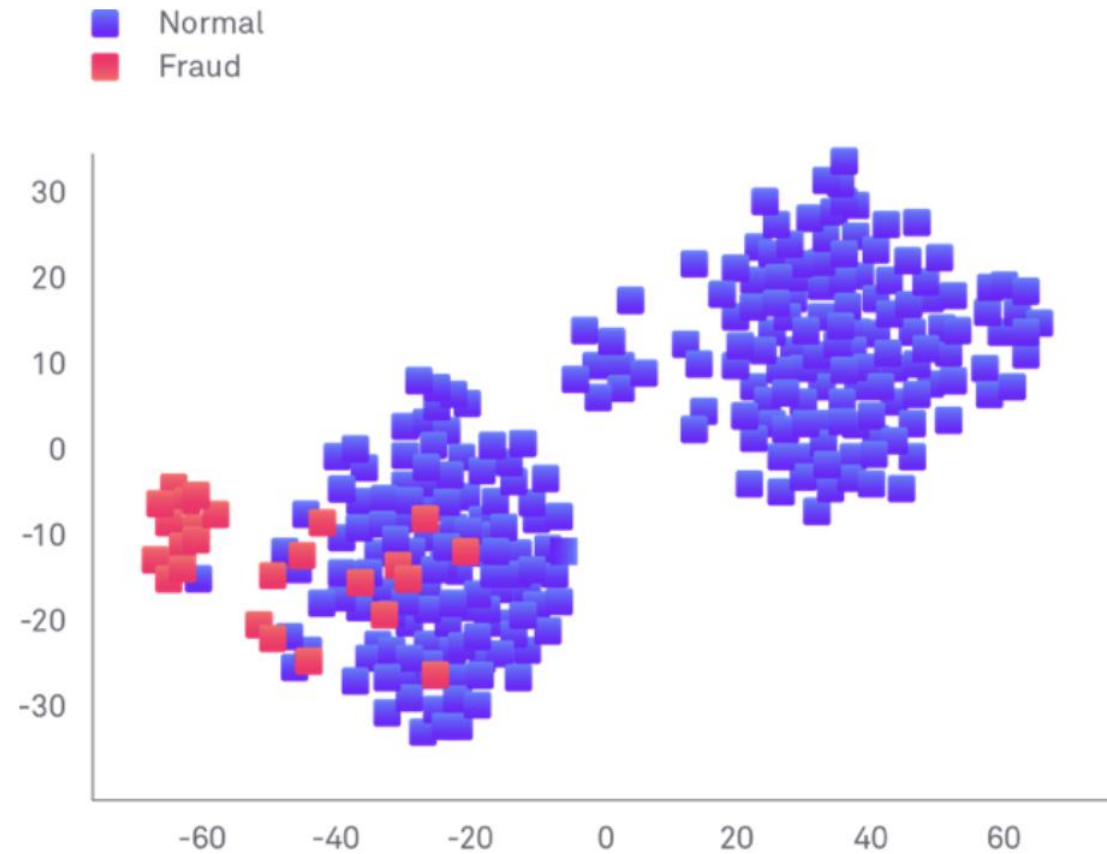


Figure 1. Data projection distribution on a transaction classification task



Three challenges with imbalanced data

- **Bias:** Imbalanced data can lead to model bias, where the model becomes overly influenced by the majority class. It may struggle to make accurate predictions for the minority class.
- **High Accuracy, Low Performance:** A model trained on imbalanced data may appear to have high accuracy but may perform poorly on minority classes, which are often the ones of greater interest.
- **Missed Insights:** Data imbalance can result in the loss of important insights and patterns present in the minority class, leading to missed opportunities or critical errors.



You trained a model to predict cancer from image data using a state of the art *Hierarchical siamese CNN* with dynamic kernel activations...

Your model has an accuracy of 99.9%



An imbalanced classification problem is an example of a classification problem where the distribution of examples across the known classes is biased or skewed.



Heart disease dataset

One way to do this is by creating a **Confusion Matrix** for each method.

		Actual	
		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease		
	Does Not Have Heart Disease		



Heart disease dataset

...then the top left corner contains
True Positives.

Actual			
		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease	True Positives	
	Does Not Have Heart Disease		

TRUE POSITIVE



TRUE NEGATIVE

These are the patients that *did not have heart disease* that were correctly identified by the algorithm.

		Actual	
		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease	True Positives	
	Does Not Have Heart Disease		True Negatives

FALSE POSITIVE

False Positives are patients that do not have heart disease, but the algorithm says they do.

		Actual	
		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease	True Positives	False Positives
	Does Not Have Heart Disease	False Negatives	True Negatives

False Negatives are when a patient has heart disease, but the algorithm said they didn't.

		Actual	
		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease	True Positives	
	Does Not Have Heart Disease	False Negatives	True Negatives

FALSE NEGATIVE

After plotting your class distribution you see that you have **thousands of negative examples** but just a couple of positives.





Something is Wrong!!

By looking at the confusion matrix, you realize that the model **does not detect** any of the **positive examples**.



Classifiers try to reduce the overall error so they can be biased towards the majority class.

Negatives = 998

Positives = 2

By always predicting a negative class the accuracy will be 99.8%

Your dataset is imbalanced!!!

Now What???



The Class Imbalance Problem

- Data sets are said to be balanced if there are, approximately, as many positive examples of the concept as there are negative ones.
- There exist many domains that have **unbalanced data sets**.
- Examples:
 - a) Helicopter Gearbox Fault Monitoring
 - b) Discrimination between Earthquakes and Nuclear Explosions
 - c) Document Filtering
 - d) Detection of **Oil Spills**
 - e) Detection of **cancerous cells**
 - f) Detection of **Fraudulent** Telephone Calls
 - g) Detection of **hotspots** in ASIC/FPGA Placements
 - h) Detection of **unrouteable** designs in VLSI
 - i) **Fraud and default** prediction
 - j) **Mail Spam** Detection



The Class Imbalance Problem

- The problem with class imbalances is that standard learners are often biased towards the majority class.
- That is because these classifiers attempt to reduce global quantities such as the error rate, not taking the data distribution into consideration.
- As a result, examples from the overwhelming class are well-classified whereas examples from the minority class tend to be misclassified.



The Class Imbalance Problem

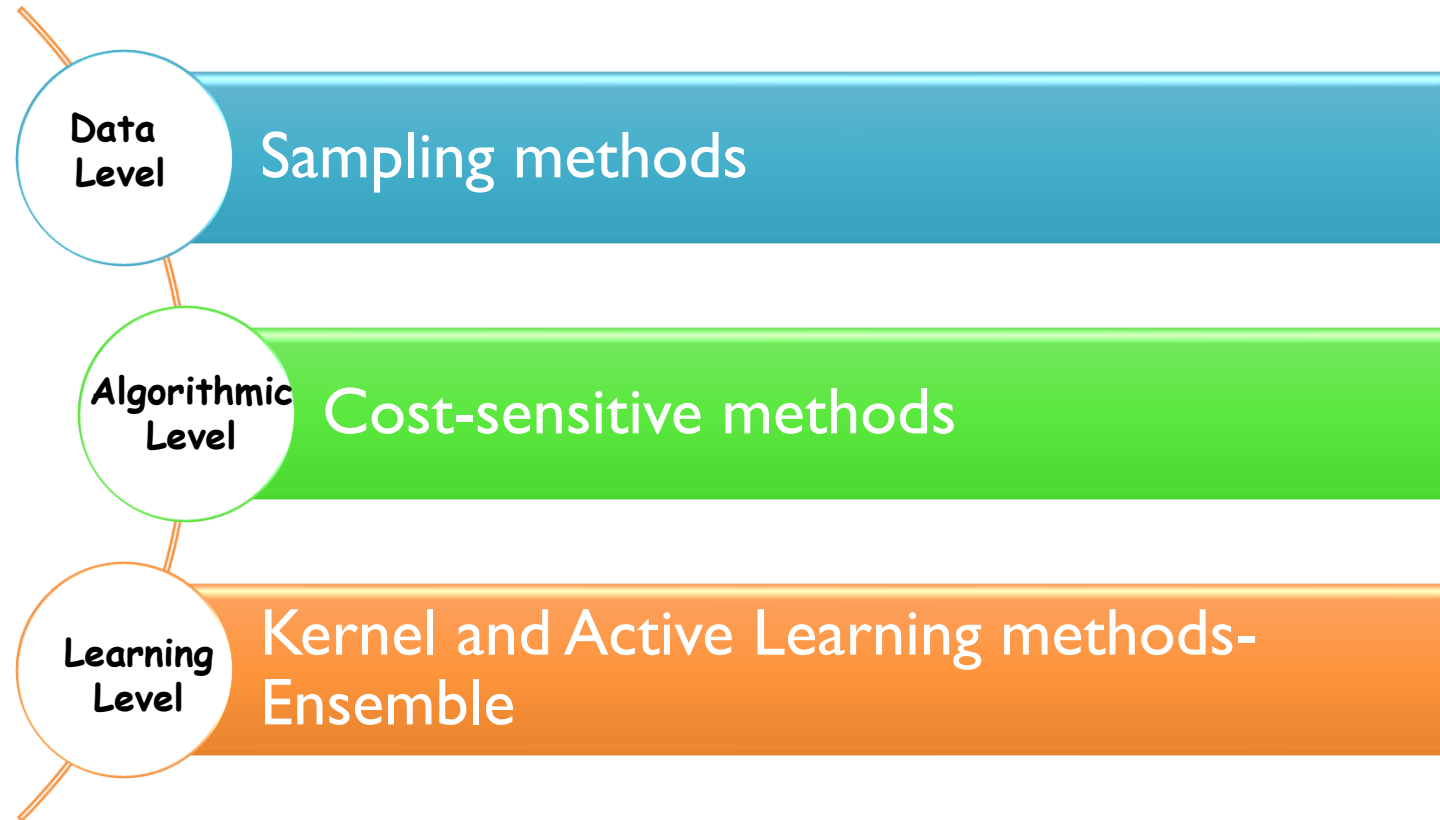
- For classification problems, we often use accuracy as the evaluation metric.
- It is easy to calculate and intuitive:
 - Accuracy = # of correct predictions / # of total predictions
- But, it is misleading for highly imbalanced datasets!!
- For example, in credit card fraud detection, we can set a model to always classify new transactions as legit.
- The accuracy could be high at 99.0% if 99.0% in the dataset is all legit.
- But, don't forget that our goal is to detect fraud, so such a model is useless.



Solutions



Solutions to Imbalanced Learning



Several Common Approaches

- At the data Level: Re-Sampling
 - Oversampling (Random or Directed)
 - Add more examples to minority class
 - Undersampling (Random or Directed)
 - Remove samples from majority class
- At the Algorithmic Level:
 - Adjusting the Costs or weights of classes
 - Adjusting the decision threshold / probabilistic estimate at the tree leaf

Most of the machine learning models provide a parameter called class weights

Several Common Approaches

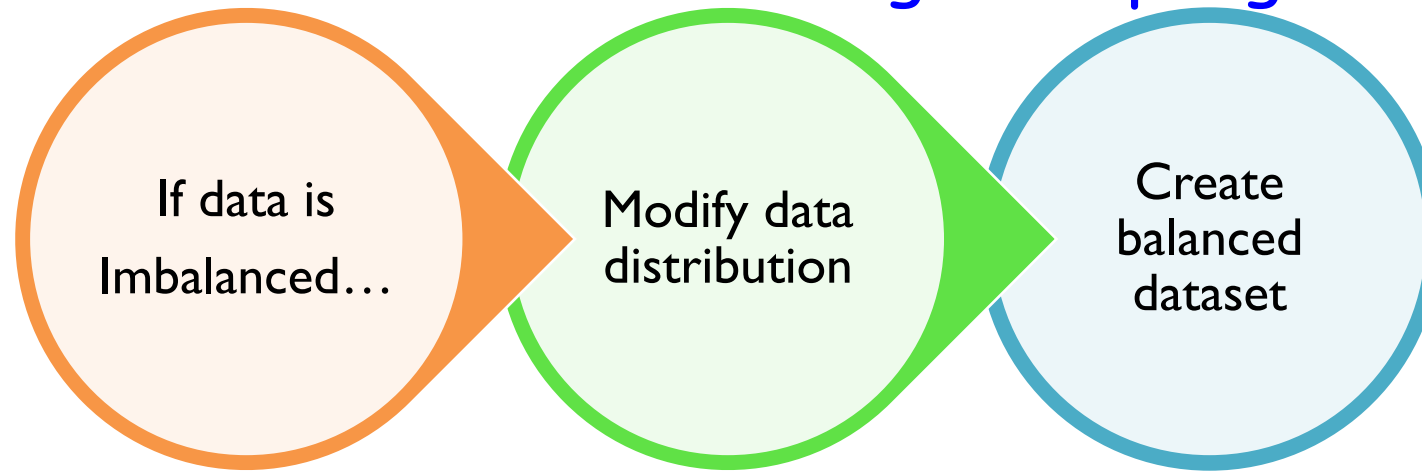
➤ At the Learning Level:

change *how the model learns* so that it pays *extra attention* to difficult and minority-class samples during training, instead of treating **all samples equally**.



Sampling Methods

Create balance through sampling



A widely adopted technique for dealing with highly unbalanced datasets is called resampling.

1. Removing samples from the majority class (under-sampling).
2. Adding more examples to the minority class (over-sampling).
3. Or perform both simultaneously:
 - Under-Sample the majority &
 - Over-Sample the minority



Sampling Methods

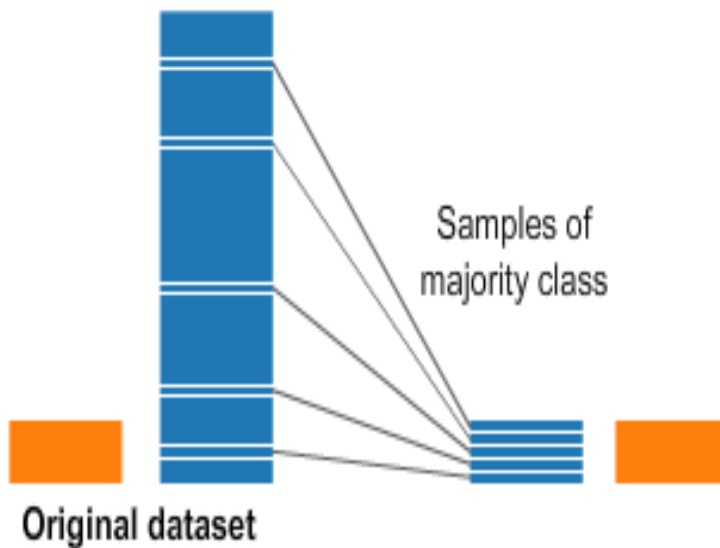
Create balance though sampling

Oversampling may just randomly replicate records within the dataset!!

Can cause loss of information.

Can cause overfitting!

Undersampling



Oversampling



Advantages and disadvantages of Under-sampling and Oversampling?

Advantages of oversampling for machine learning

- Does not decrease the size of the dataset.
- Does not lose any information

Disadvantages of oversampling for machine learning

- Can increase the chance of overfitting.
- One disadvantage of oversampling for machine learning is that it can increase the chance of your model **overfitting** (model memorizes training data but performs poor on testing data)to your training data.
- Whether you perform oversampling by introducing duplicated observations or synthetic observations that closely resemble existing observations, **you are increasing the presence of characteristics in your dataset** that were originally only applicable to a small number of observations.
- These characteristics might be specific details of a few specific observations rather than broadly applicable trends.



Advantages of under sampling for machine learning

- No need to introduce redundant information into your dataset.

Disadvantages of under sampling for machine learning

- Reduces the size of your dataset.
- Loses information.



SMOTE Synthetic Minority Oversampling Technique



SMOTE: Resampling Approach

- **SMOTE** stands for:
Synthetic **M**inority **O**versampling **T**echnique
- It is a technique designed by Hall et. al in 2002.
- **SMOTE** is an oversampling method that synthesizes new plausible examples in the minority class.
- **SMOTE** not only increases the size of the training set, but also increases the variety!!
- **SMOTE** currently yields the best results as far as re-sampling and modifying the probabilistic estimate techniques go (Chawla, 2003).



SMOTE's Informed Oversampling Procedure

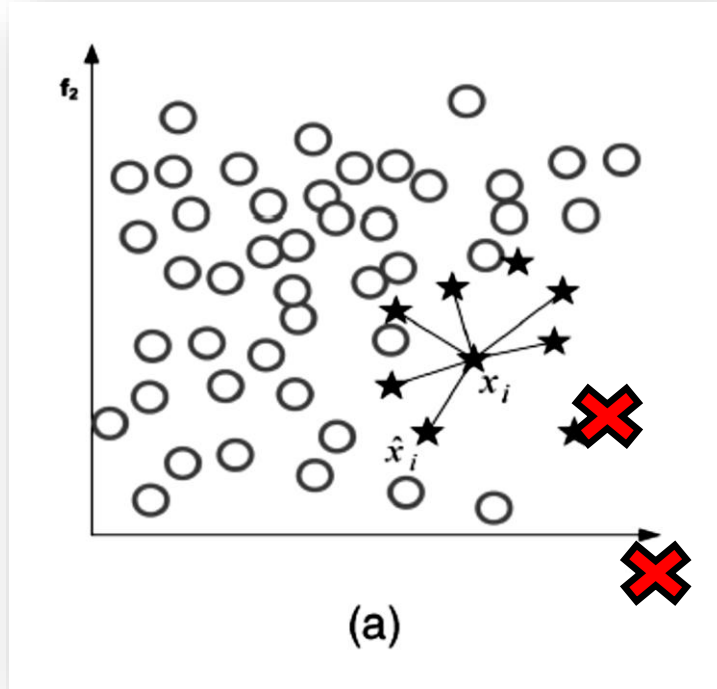
For each **Minority Sample**

- I. Find its **k-nearest** minority neighbors
 - II. Randomly select **j** of these neighbors
 - III. Randomly generate synthetic samples along the lines joining the **minority sample** and its **j** selected neighbors
- (**j** depends on the amount of oversampling desired)

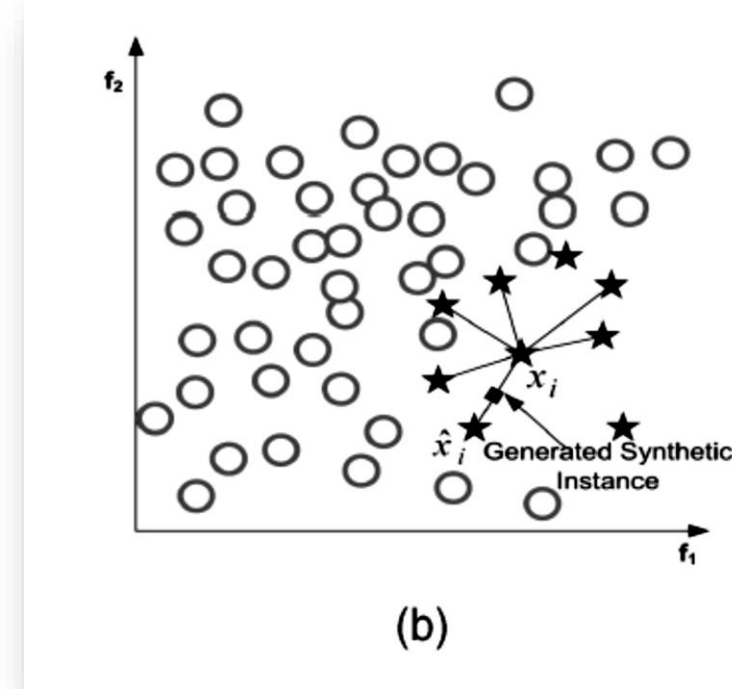
For instance, if it sees two examples (of the same class) near each other, it creates a third artificial one, in the middle of the original two.



Synthetic Minority Oversampling Technique (SMOTE)



- Find its k -nearest minority neighbors
- Randomly select j of these neighbors



- Randomly generate synthetic samples along the lines joining the minority sample and its j selected neighbors



SMOTE's Shortcomings

- Overgeneralization

- a) SMOTE's procedure may **blindly generalize** the minority area **without regard** to the majority class.
- b) It may oversample **noisy samples**
- c) It may oversample **uninformative samples**

- Lack of Flexibility (i.e., Static)

- a) The number of synthetic samples generated by SMOTE is **fixed in advance**, thus not allowing for any flexibility in the re-balancing rate.
- b) It would be nice to increase the minority class just to the right value (i.e., **not excessive**) to avoid the **side affects** of unbalanced datasets



```

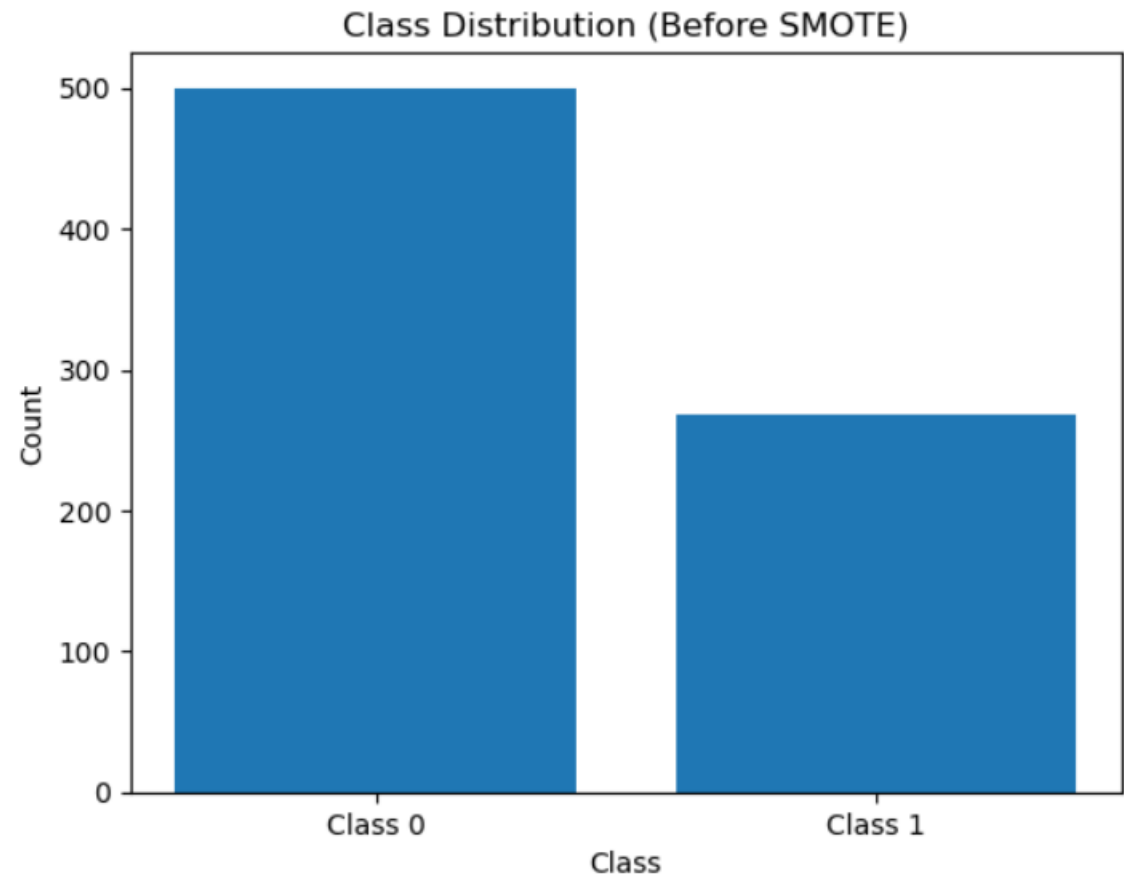
import matplotlib.pyplot as plt
import pandas as pd
from imblearn.over_sampling import SMOTE

data = pd.read_csv("C:/Users/GAYATHRI/Downloads/diabetes (1).csv")
X = data.drop("Outcome", axis=1)
y = data["Outcome"]

count_class = y.value_counts()
plt.bar(count_class.index, count_class.values)
plt.xlabel('Class')
plt.ylabel('Count')
plt.title('Class Distribution (Before SMOTE)')
plt.xticks(count_class.index, ['Class 0', 'Class 1'])
plt.show()

smote = SMOTE(sampling_strategy='minority')
X_res, y_res = smote.fit_resample(X, y)
print("After SMOTE:\n", y_res.value_counts())

```



After SMOTE:

Outcome	Count
1	500
0	500

Name: count, dtype: int64

Cost-Sensitive Approach



Cost-Sensitive Approach

- Cost-sensitive learning is a subfield of machine learning that takes the **costs of prediction errors (minimize total misclassification cost)** (and potentially other costs) into account when training a machine learning model.
- Log. regression **doesn't support** imbalanced classification directly.
- Instead, the training algorithm used to fit the log. regression model **must be modified** to take the skewed distribution into account.
- This can be achieved by specifying a class weighting configuration that is used to influence the amount that logistic regression coefficients are updated during training.
- The weighting can penalize the model less for errors made on examples from the majority class and penalize the model more for errors made on examples from the minority class.
- The result is a version of logistic regression that performs better on imbalanced classification tasks, generally referred to as cost-sensitive or weighted logistic regression.

<https://machinelearningmastery.com/cost-sensitive-logistic-regression/>

Cost-Sensitive Approach

- In Logistic regression, we calculate loss per example using binary cross-entropy:

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

- Loss = $-y \log(p(y)) - (1-y) \log(1-p(y))$
 - where y is the label (1 for class A and 0 for class B)
 - $p(y)$ is the predicted probability of the point being class A.
- In this particular form, we give equal weight to both the positive and the negative classes.
 - However, if we set class_weight as class_weight = {0:1, 1:20}, the classifier in the background tries to minimize:

That means we penalize our model around 20 times more when it misclassifies a positive minority example in this case.

$$\text{NewLoss} = -20 * y \log(p(y)) + 1 * (1-y) \log(1-p(y))$$



**Handling Imbalanced Classes
using
`class_weight = 'balanced'`**



Data Visualization



Data visualization

- What is Data Visualization?
- Data visualization is a graphical representation of data and information.
- With the help of data visualization, we can see how the data looks like and what kind of correlation is held by the attributes of the data.
- It is the fastest way to see if the features correspond to the output.



Importance of Data Visualization in Machine Learning

- **Exploring Data** — Data visualization is an essential tool for exploring and understanding data. Visualization can help to identify patterns, correlations, and outliers and can also help to detect data quality issues such as missing values and inconsistencies
- **Feature Selection** — Data visualization can help to select relevant features for the ML model. By visualizing the data and its relationship with the target variable, you can identify features that are strongly correlated with the target variable and exclude irrelevant features that have little predictive power.
- **Model Evaluation** — Data visualization can be used to evaluate the performance of the ML model. Visualization techniques such as ROC curves, precision-recall curves, and [confusion matrices](#) can help to understand the accuracy, [precision, recall](#), and F1 score of the model.
- **Communicating Insights** — Data visualization is an effective way to communicate insights and results to stakeholders who may not have a technical background.



Popular Python Libraries for Data Visualization

Matplotlib

- [Matplotlib](#) is one of the most popular Python packages used for data visualization. It is a cross-platform library for making 2D plots from data in arrays. It provides an object-oriented API that helps in embedding plots in applications using Python GUI toolkits such as PyQt, VxPython, or Tkinter

Seaborn

- [Seaborn](#) is an open source, BSD-licensed Python library providing high level API (High-level API= less code, better visuals-Heatmap, pair plot) for visualizing the data using Python programming language

Plotly

- [Plotly](#) is a Montreal based technical computing company involved in development of data analytics and visualisation tools such as Dash and Chart Studio - **interactive and web-based visualizations.**

Bokeh

- [Bokeh](#) is a data visualization library for Python. Unlike Matplotlib and Seaborn, they are also Python packages for data visualization, Bokeh renders its plots using HTML and JavaScript - **browser-based visualization.**



DEMO IN PYTHON using iris flower data set



Exploratory data analysis: Visualization before analysis, visualizing a single variable, Examining multivariate

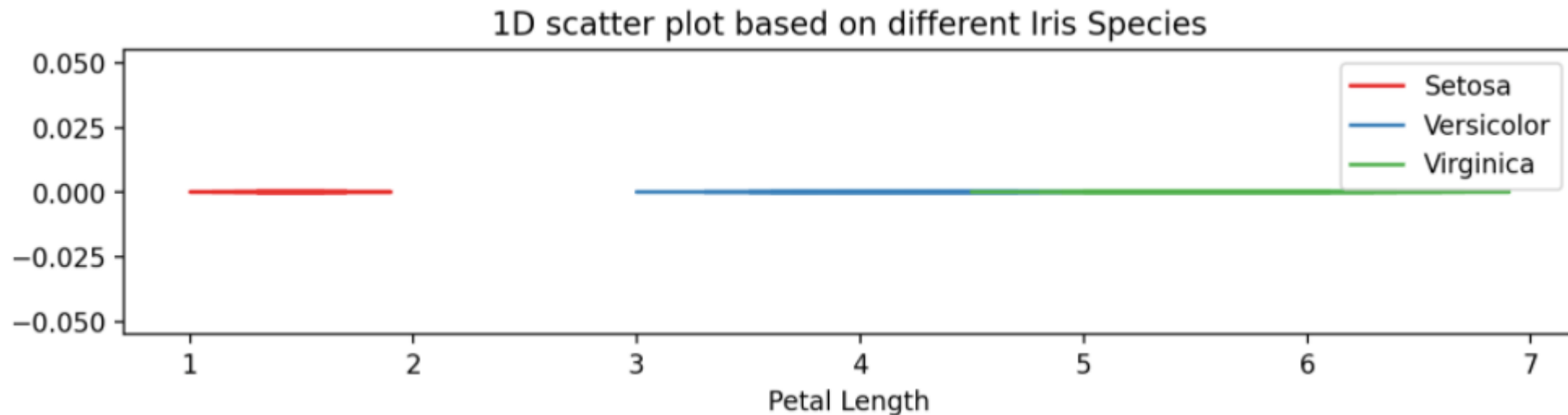


- Exploratory Data Analysis (EDA) is a visual method for finding structures in data with the power of the human eye or brain that can easily detect structures or process the patterns.
- However, it is very difficult when the dataset is larger. EDA is essential before going to apply actual machine learning algorithms for the different kinds of predictions.
- It is a simple dataset containing three flowers of the Iris species family, and these are [Iris setosa](#), [Iris virginica](#), and [Iris versicolor](#).
- Four features are selected and measured in centimeters per flower, and the four features are
- sepal length, sepal width, petal length, and petal width.
- The selection of features was completely subjective to the domain expert.
- There are fifty samples per Iris species family
- The objective of using this dataset is to **classify a new flower with a given four features belonging to one of the three classes.**



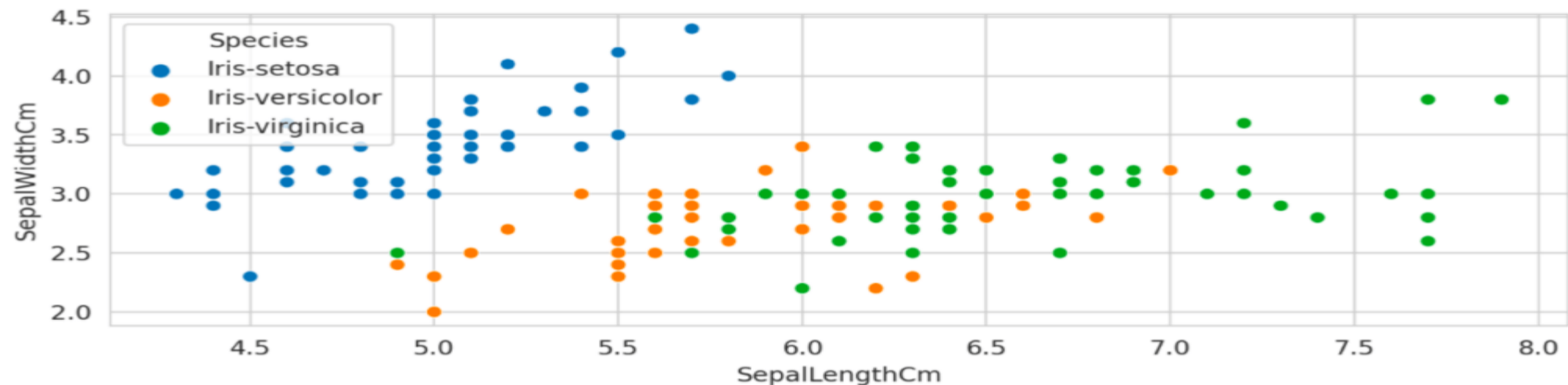
Univariate Analysis

- Univariate analysis performs on a variable to find the range of parameters that will be helpful to **understand a variable**.
- These parameters are the central tendency & spread. To find a central tendency, we can go for mean, median, or mode to select the appropriate method to find the central tendency once we understand the data.



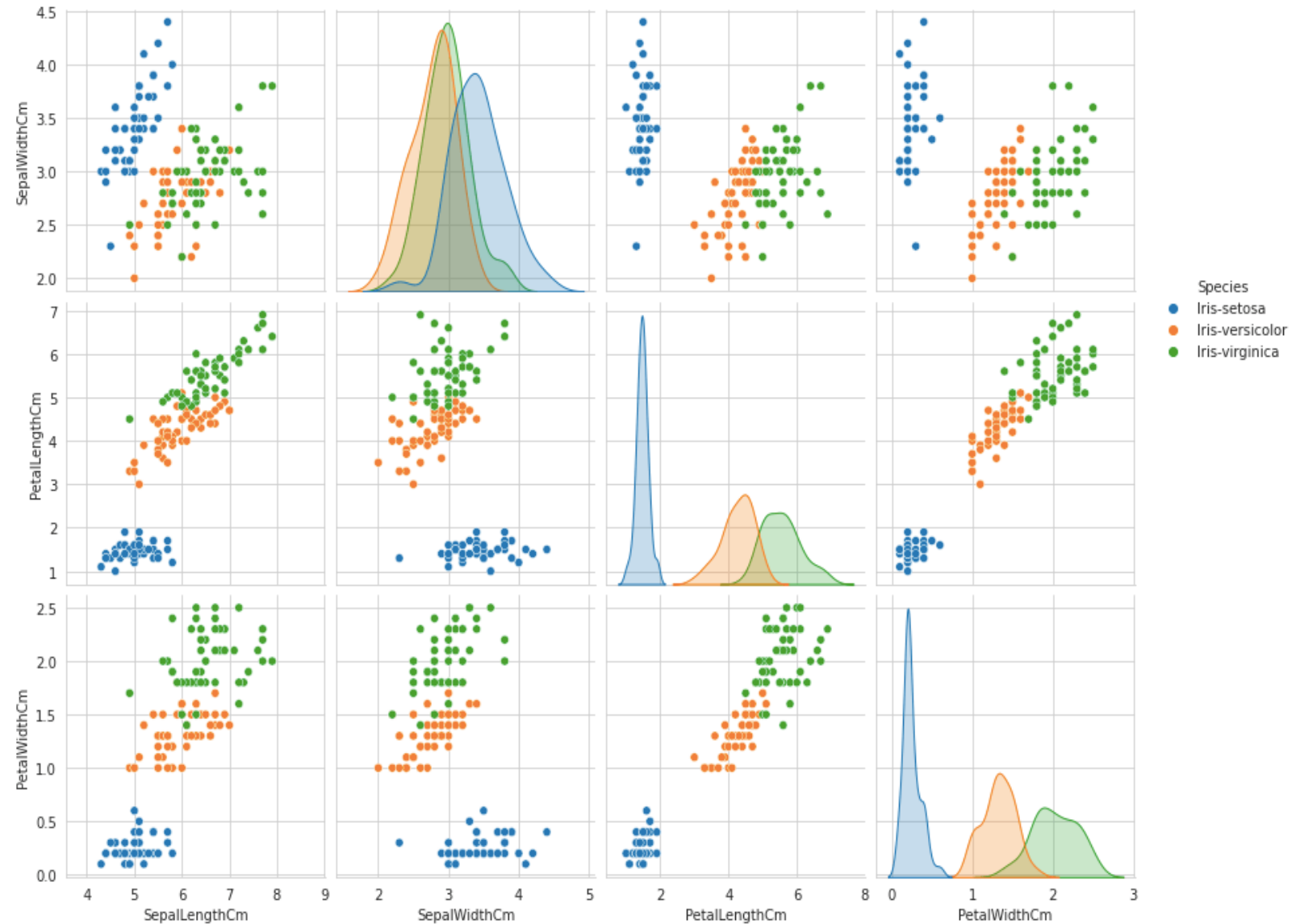
Bivariate analysis

- **Bivariate analysis** is one of the simplest ways of quantitative **analysis**.
- It involves the **analysis of two variables** for the purpose of determining relationship between them.
- **Scatter Plot**
- The Scatter plot is the 2D plot the represents the relationship between two variables.
- We will use different combinations of features and analyze which one produces a better scatter plot than the other from the point of analysis in the 2D plane.



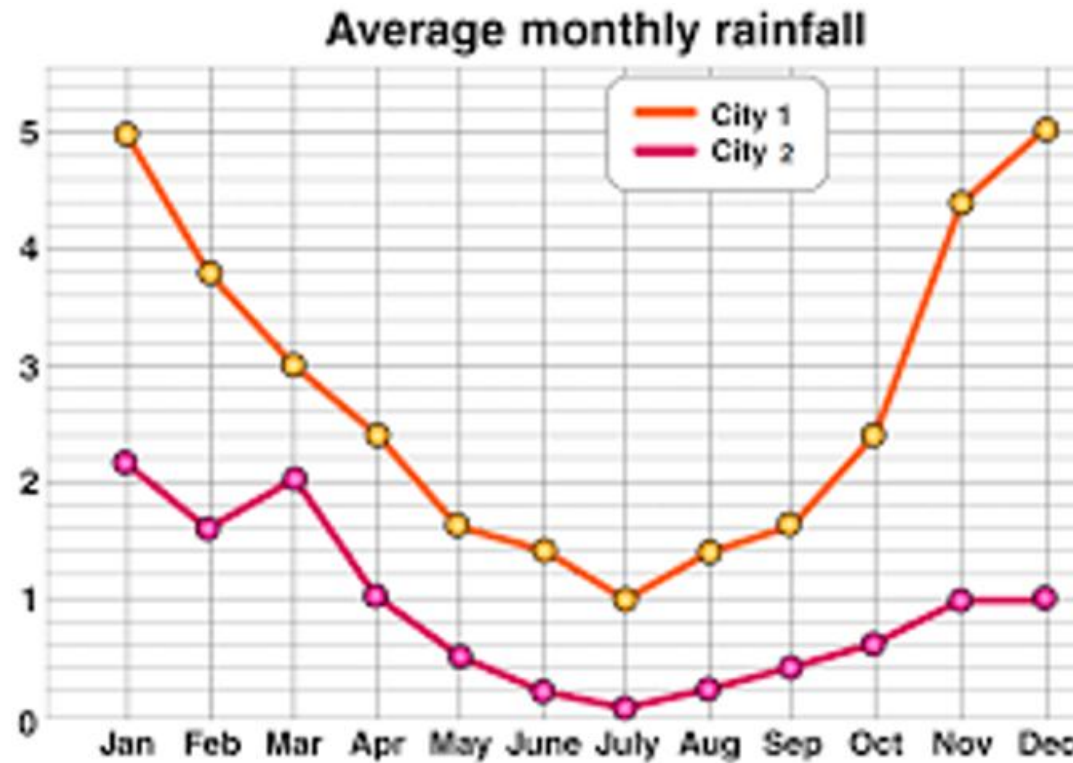
Multivariate Analysis

- Pair plots are mainly used for multivariate analysis.
- Visualize correlations between numerical variables.
- Understand distributions of individual features.
- Identify patterns in the data.
- Separate classes using the hue parameter (color-coding).



Line Chart

- A line chart, also known as a line graph or curve chart, is a graphical representation used to display data points connected by straight lines.
- This type of chart is particularly useful for visualizing trends, changes, and relationships in data over a continuous interval, often time



- A line chart uses points connected by line segments from left to right to demonstrate changes in value.
- The horizontal axis depicts a continuous progression, often that of time, while the vertical axis reports values for a metric of interest across that progression.
- a line chart when you want to emphasize changes in values for one variable (plotted on the vertical axis) for continuous values of a second variable (plotted on the horizontal).
- continuous data over a specific period.



Types of Data That Can Be Displayed Using Line Charts

- **Time Series Data**

- Line charts are particularly effective for showcasing time series data, where data points are collected and recorded at regular intervals over time. **This type of data reveals trends, patterns, and seasonality, making it perfect for line charts**
- Examples include stock prices, temperature variations, sales figures, and website traffic data.

- **Sequential Data**

- When data points are related in a sequential order, a line chart can help depict how values progress over the sequence.
- **Comparison of Multiple Data Sets**



Histogram

- A histogram is a chart that plots the distribution of a numeric variable's values as a series of bars.
- Each bar typically covers a range of numeric values called a bin or class;
- a bar's height indicates the frequency of data points with a value within the corresponding bin.
- **Components of a Histogram**
 - A histogram is a type of bar chart where each bar represents the frequency or count of data points falling within a certain range or bin. It is particularly useful for understanding the distribution, central tendency, and spread of the data. histograms can be drawn when data are in the form of the frequency distribution of a continuous series
 - **Bins (Class Intervals or Cells):** These are the ranges into which data is grouped. Each bin covers a range of values and is represented by a bar in the histogram. The width of each bin can be uniform or vary based on the chosen range.
 - **Frequency:** This is the count of data points that fall within each bin. The height of each bar in the histogram represents this count.



Frequency Distribution (equal intervals)

Weight	frequency
100-110	8
110-120	16
120-130	12
130-140	8
140-150	6

Sum of frequency = 50

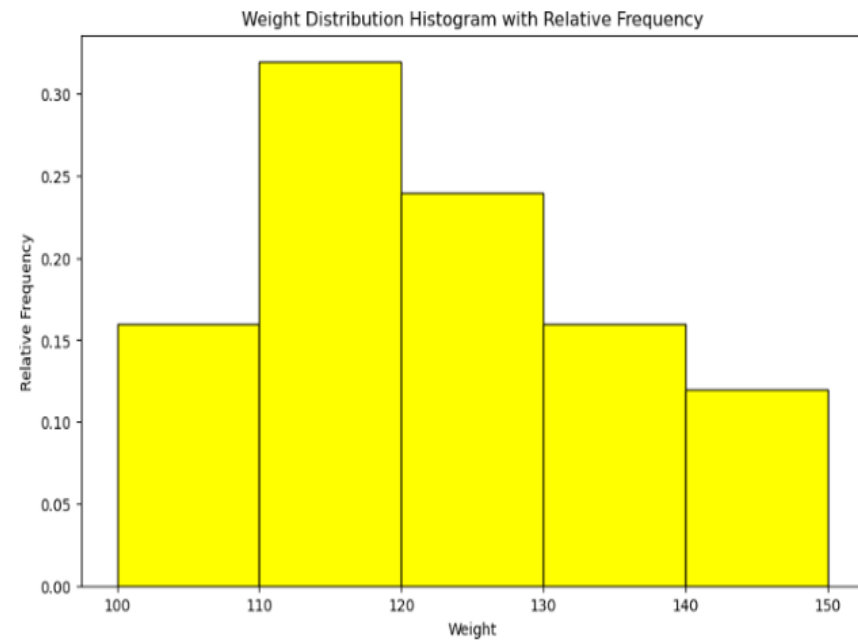
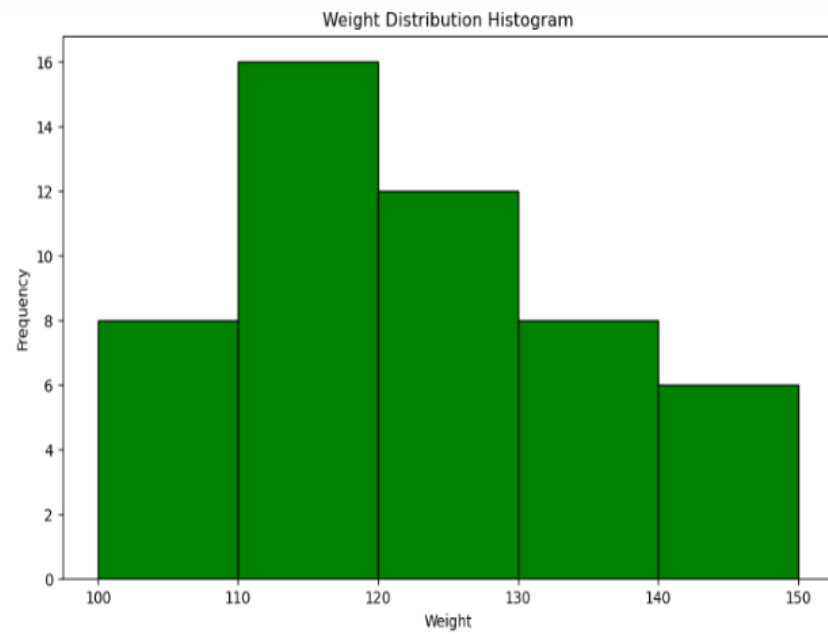
frequency	Relative frequency
8	$8/50=0.16$
16	$16/50=0.32$
12	$12/50=0.24$
8	$8/50=0.16$
6	$6/50=0.12$

Relative Frequency Distribution

Weight	Relative frequency
100-110	0.16
110-120	0.32
120-130	0.24
130-140	0.16
140-150	0.12

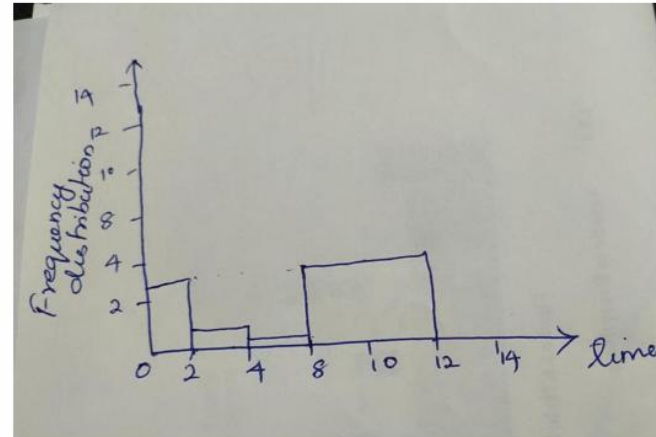
Relative Sum of frequency = 1





- It represents the **height of the rectangle** in a histogram when the class widths are **not equal**. This ensures the **area** of each rectangle represents the **frequency**, not the height alone.
- Frequency density or rectangular height = $\text{Freq} / \text{interval or class width}$

time	frequency	Frequency density
$0 < t \leq 2$	5	$5/2 = 2.5$
$2 < t \leq 4$	3	$3/2 = 1.5$
$4 < t \leq 8$	2	$2/4 = 0.5$
$8 < t \leq 12$	10	$10/4 = 2.5$



Scatter

- A scatter plot uses dots to represent values for two different numeric variables. The position of each dot on the horizontal and vertical axis indicates values for an individual data point.
- Scatter plots are used to observe relationships between variables.
- If the variables are correlated, the points will fall along a line or curve

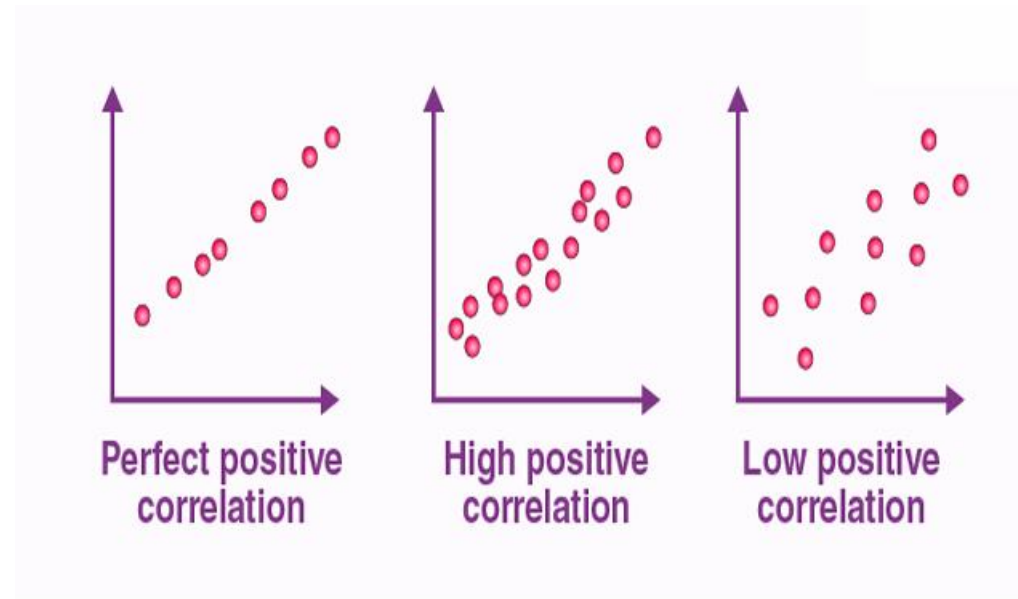
Types of correlation

- The scatter plot explains the correlation between two attributes or variables. It represents how closely the two variables are connected.
- There can be three such situations to see the relation between the two variables
 - 1.Positive Correlation
 - 2.Negative Correlation
 - 3.No Correlation



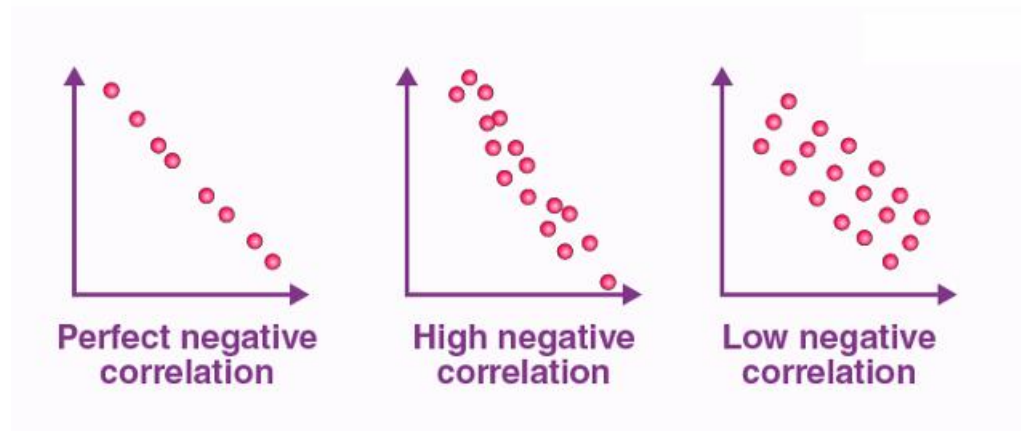
Positive Correlation

- Perfect Positive—Which represents a perfectly straight line
- High Positive—All points are nearby
- Low Positive—When all the points are scattered



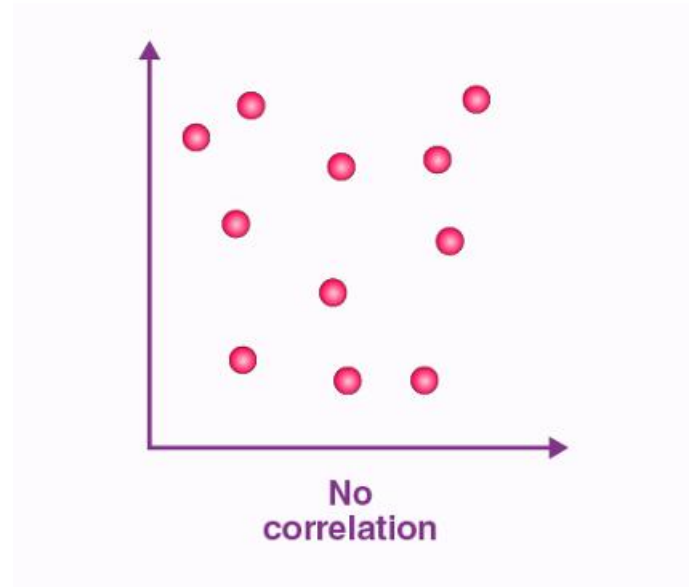
Negative Correlation

- When the points in the scatter graph fall while moving left to right, then it is called a negative correlation. It means the values of one variable are decreasing with respect to another.
- Perfect Negative—Which form almost a straight line
- High Negative—When points are near to one another
- Low Negative—When points are in scattered form



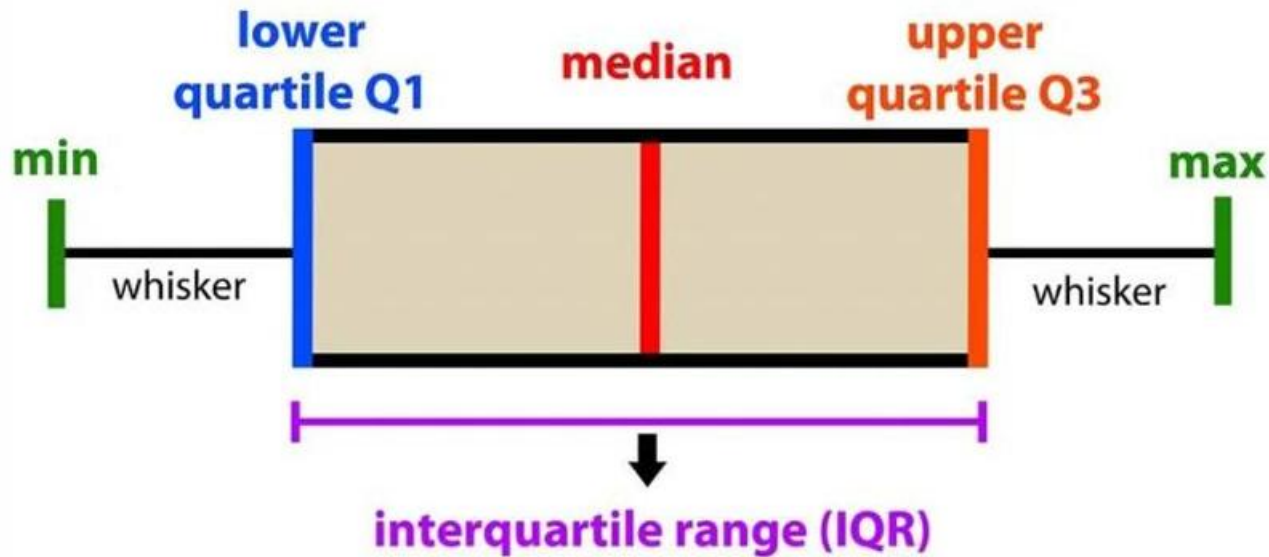
No Correlation

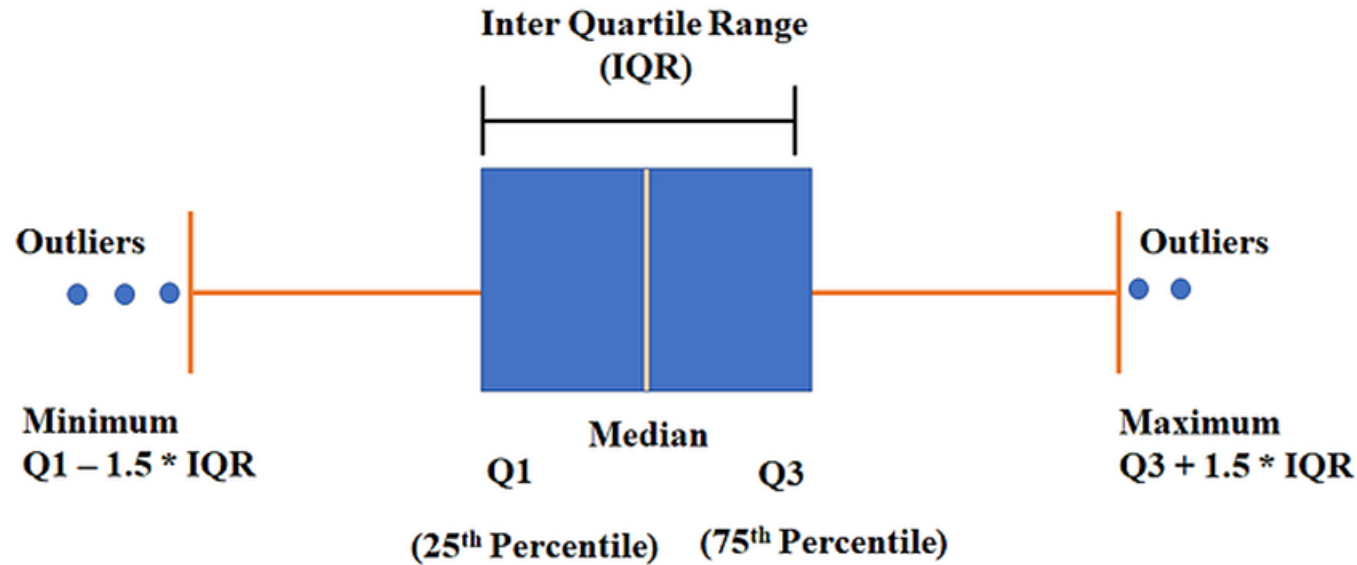
- When the points are scattered all over the graph and it is difficult to conclude whether the values are increasing or decreasing, then there is no correlation between the variables.



BOX PLOT

- In descriptive statistics, a box plot or boxplot (also known as a box and whisker plot) is a type of chart often used in explanatory data analysis.
- Box plots visually show the distribution of **numerical data and skewness by displaying the data quartiles percentiles) and averages.**





$$IQR = Q3 - Q1 = 40 - 20 = 20$$

- Lower boundary:

$$Q1 - 1.5 \times IQR = 20 - 1.5 \times 20 = 20 - 30 = -10$$

- Upper boundary:

$$Q3 + 1.5 \times IQR = 40 + 1.5 \times 20 = 40 + 30 = 70$$



Use-Cases of Box Plot

- Box plots provide a visual summary of the data with which we can quickly identify the average value of the data, how dispersed the data is, whether the data is skewed or not (skewness).
- The Median gives you the average value of the data.
- Box Plots shows Skewness of the data
 - a) If the Median is at the center of the Box and the whiskers are almost the same on both the ends then the data is Normally Distributed.
 - b) If the Median lies closer to the First Quartile and if the whisker at the lower end is shorter then it has a Positive Skew (**Right Skew**).
 - c) If the Median lies closer to the Third Quartile and if the whisker at the upper end is shorter than it has a Negative Skew (**Left Skew**).



Normal Distribution



Positive Skew



Negative Skew



Heat map

- A heat map is a two-dimensional representation of data various values are represented by colors
- Heatmap data visualization is a technique that uses color to represent data values.
- The data is typically arranged in a grid or matrix format, with each cell assigned a color based on its value.
- The **intensity of the color corresponds to the magnitude of the data**, allowing viewers to discern patterns and trends at a glance.
- Heatmaps are particularly useful for visualizing large datasets and identifying areas of interest or concentration.



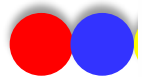
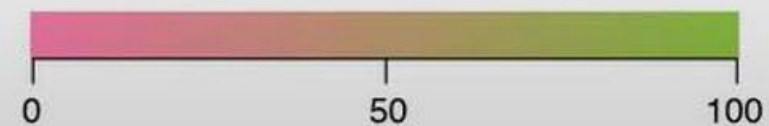
Based on categorical data

	A	B	C
D	Low	High	Medium
E	Medium	High	Low
F	High	Medium	Low

Low
Medium
High

Based on numerical data

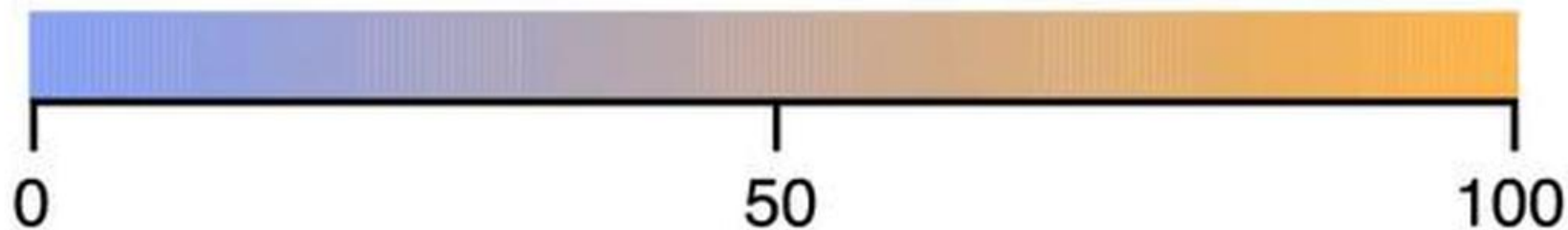
	A	B	C
D	12	48	81
E	97	79	10
F	5	43	83



Alternative value scale broken into ranges:



Value scale for determining cell colouring:



Temperature Scale:

